pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Bag of Words and TF-IDF

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

From tokens to numbers

1. Count

Bag of words: one row per document, one column per token

2. Weight

TF-IDF: each count times how rare the word is

3. Compare

Cosine similarity: one number for two documents

```
textkit.term_counts(docs)          textkit.idf(doc_freq, n_docs)
textkit.tfidf_by_hand(counts)      textkit.cosine_similarity_matrix(A, B)
```

Each one tested against scikit-learn to 1e-12, on 226 statements and Article 4.

Real documents, described

An FOMC statement's output

Allowed names	Never called
count of words from the course's list	hawkish or dovish
dictionary score	a signal or a policy view
the action the statement states	a forecast of rates

An indenture's terms

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

How a bag of words works

The problem	A model needs numbers, and a statement is text
The columns	Learn the vocabulary: every distinct token across all documents is one column
The rows	For each document, count how often each column's token occurs; the order is thrown away
You control	The token pattern, which columns to keep, and counts or presence
What comes out	A document-term table, stored as a sparse matrix because most cells are 0

Bag of words

226

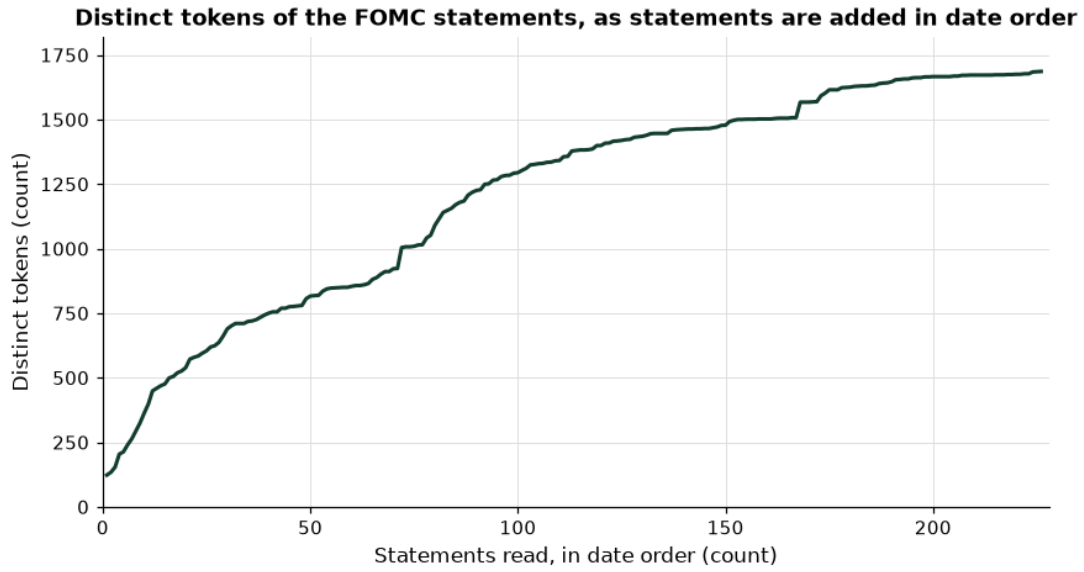
rows: one per statement

1,686

columns: one per distinct token

0.0983

share of the table not 0



The raw text gives four other columns

Only after normalize_text: and-in, banks-the. Only on the raw text: s, survey.

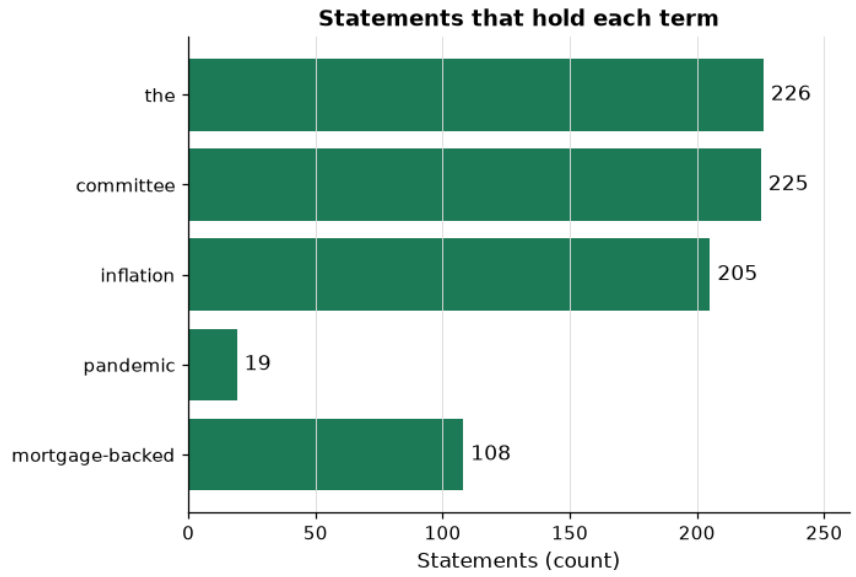
816 tokens after 50 statements, 1,294 after 100, 1,686 after all. 212 vote paragraphs left out first.

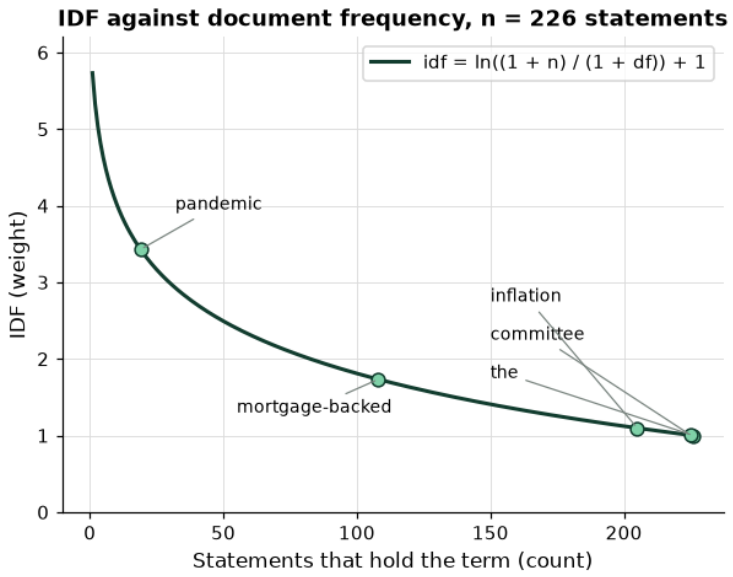
How IDF works

The problem	A word in every document cannot tell one document from another, yet it has the largest counts
Document frequency	The number of documents that contain the term
The weight	Largest for a term in one document, smallest for a term in all, through a logarithm
You control	The smoothing: 1 added to the counts, so no term divides by zero
What comes out	One weight per column of the table

Document frequency and IDF

```
idf = ln((1 + n) / (1 + df)) + 1          # scikit-learn's smooth_idf, the default
```





The two ends

In every statement (5 tokens): weight 1.0, never 0. In one only (404 tokens): the largest. pandemic, in 19: 3.4292.

A paper on term specificity

January 1972

Karen Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval", Journal of Documentation, volume 28, issue 1, pages 11 to 21.

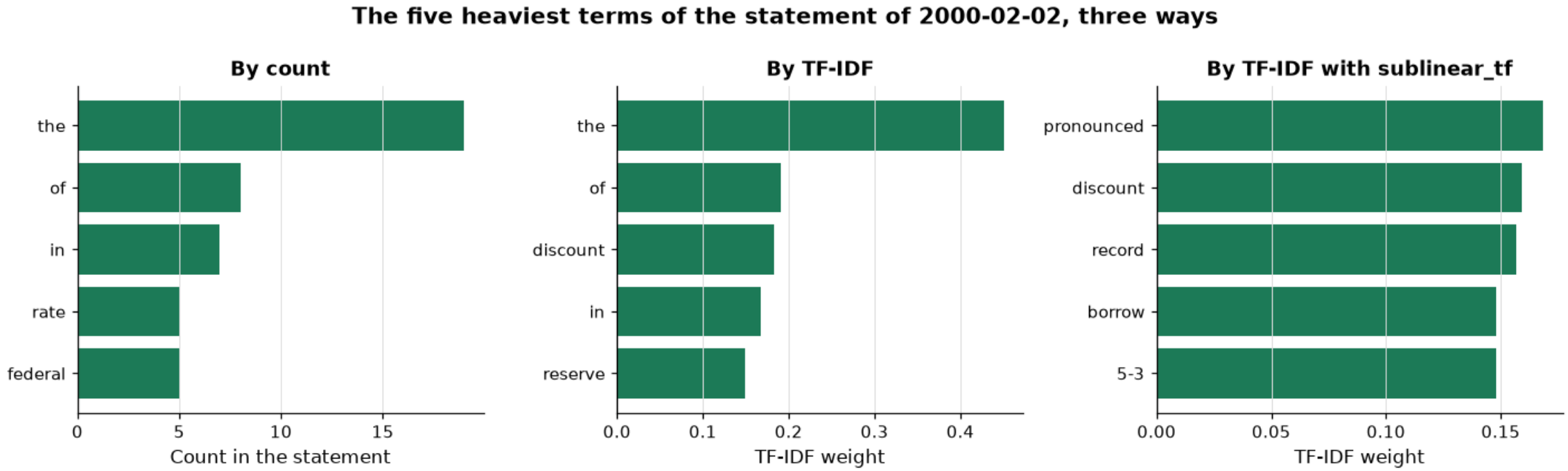
The course takes the record's facts only. The paper itself was not read for this course.

Source, read 8 October 2026: the Crossref record, api.crossref.org/works/10.1108/eb026526 (DOI 10.1108/eb026526)

How TF-IDF works

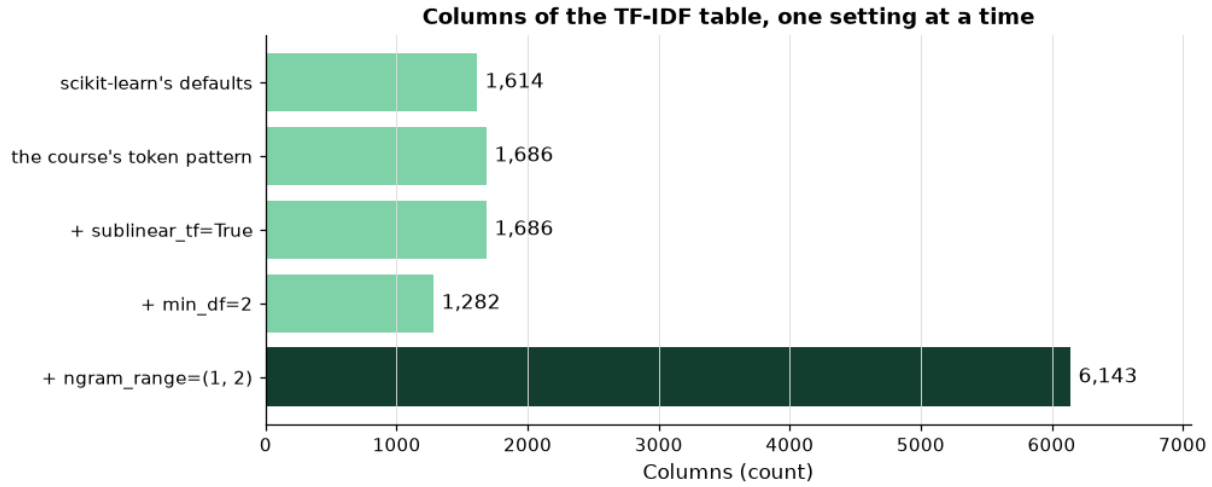
The problem	A count alone puts 'the' first and 'pandemic' last
The product	Each count times its column's IDF: frequent here and rare elsewhere weighs most
Row length	Each row is divided by its own length, so a long statement does not outweigh a short one
You control	sublinear_tf (1 plus the log of the count), the columns, and norm
What comes out	A matrix of weights, a row of length 1 per statement: the input to cosine and to day 3

TF-IDF by hand: the first statement three ways



"the" 19 times leads by count and by plain TF-IDF. With 1 + ln(tf), "pronounced" leads. Rows of length 1, equal to TfidfVectorizer within 1e-12.

One setting at a time



What each one changes

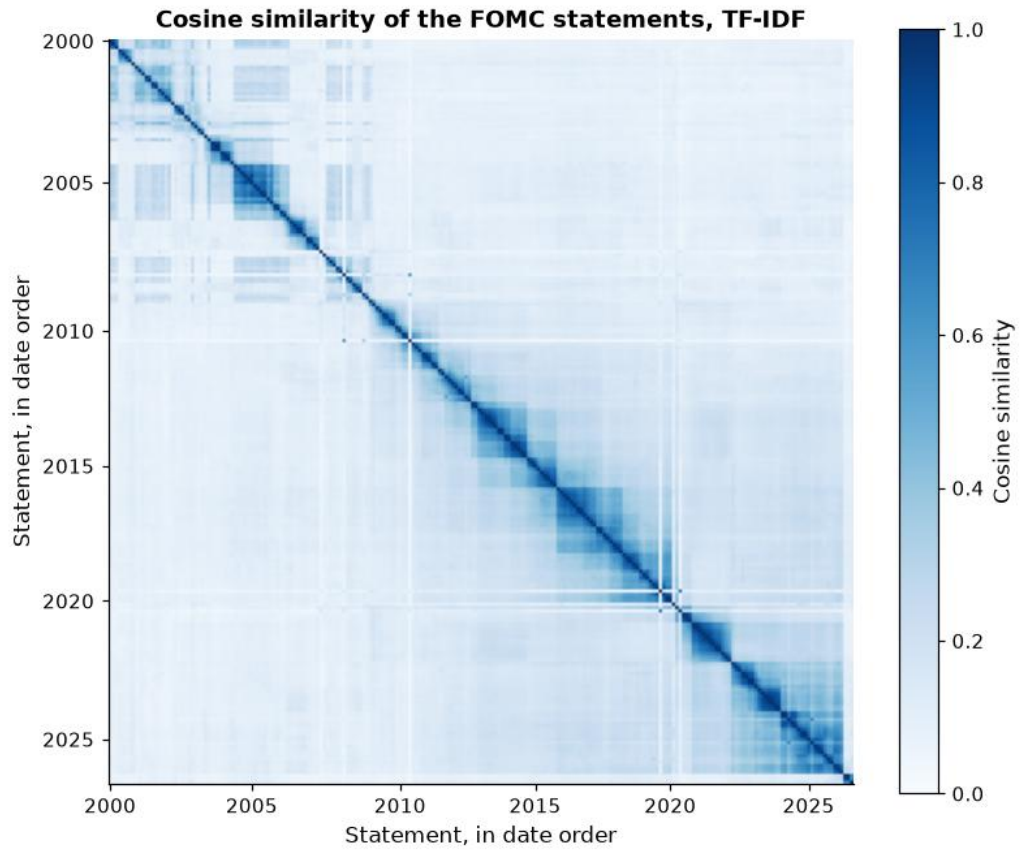
Default pattern: splits at hyphens, drops one-character tokens. Sublinear: weights only. min_df=2: drops terms in one statement. Bigrams: pairs of neighbours.

From 1,614 columns with the defaults to 6,143 with the course's setting, the dark bar.

How cosine similarity works

The problem	Alike means the same words in the same proportions, whatever the lengths
The measure	The sum of the rows' products divided by the product of their lengths
Here	Every row has length 1, so it is the sum of products
The range	1 for the same proportions, 0 for no word in common; never negative here
What comes out	One number per pair, or a square table of every pair with 1 on its diagonal

Cosine similarity



With the course's setting

Most alike: 2024-01-31, 2024-03-20 0.9699

Least alike: 2004-03-16, 2010-05-09 0.0241

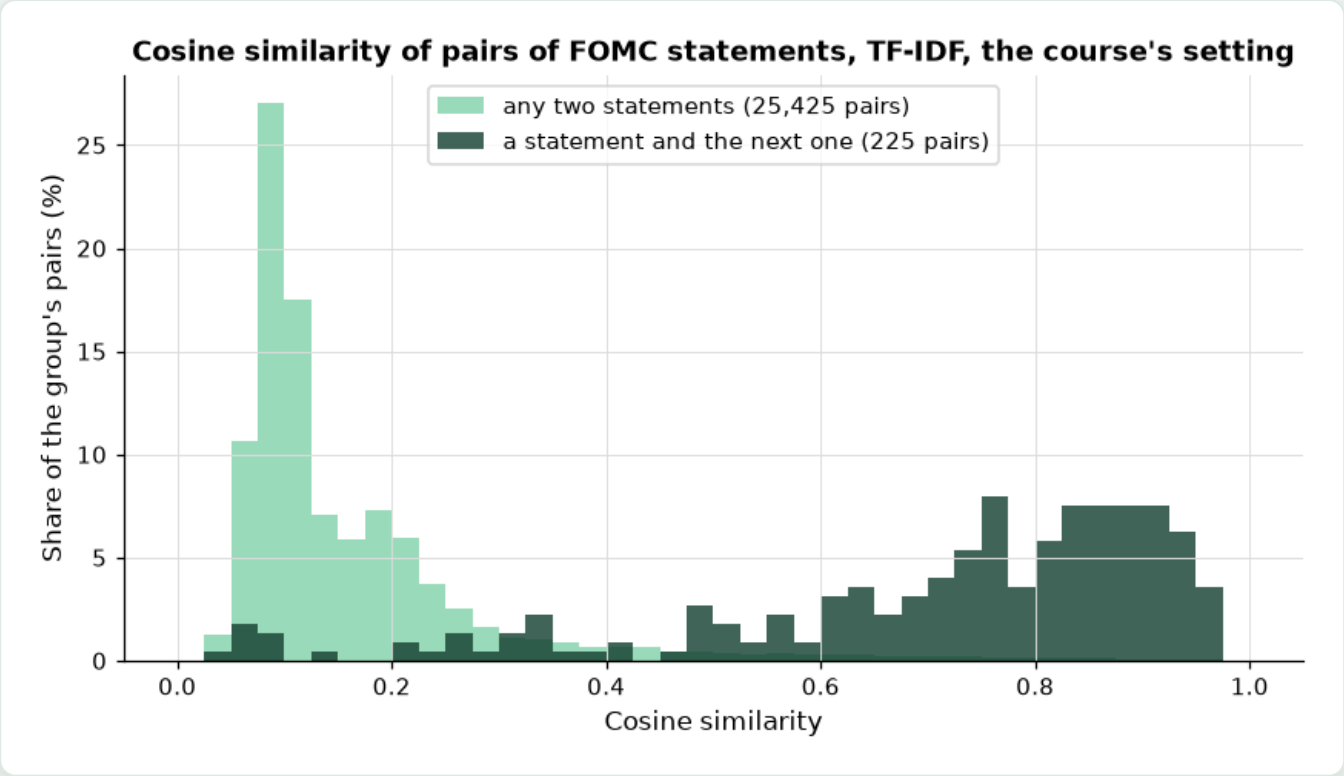
Median with the next statement: 0.7731

Median of all pairs: 0.1129

Least like the rest: 3 unscheduled releases

A cosine measures shared words. It is not a probability.

A statement and the next one



	Any two	The next one
Pairs	25,425	225
10th percentile	0.0731	0.3434
Median	0.1129	0.7731
90th percentile	0.2913	0.9239

Two humps, far apart: wording carries over from one statement to the next.

How distinctive terms are found

The problem	What sets one year's statements apart from the rest?
The difference	A term's mean TF-IDF in the year minus its mean in all the other statements
The terms	The largest differences are the year's most distinctive
You control	The weights that go in and how many terms to show
What comes out	A short list per year: a description of the wording, nothing more

Distinctive terms by year

Year	The three most distinctive terms
2008	to growth, housing contraction, labor markets
2013	asset, purchases, a highly
2014	asset, asset purchase, purchase program
2021	by at, percent for, on vaccinations
2022	war, demand imbalances, imbalances related
2026	at 3-1, 3-3, 3-3 4

How it is computed

Mean TF-IDF in the year's statements minus the mean in all others; the three largest, ties by term.

Words the statements wrote in those years, nothing more. 3-3 is 3-3/4 cut at the slash.

Article 4 as twenty documents

9 of 20

sections that hold "indebtedness"

1.7419

its idf (the largest possible: 3.3514)

67

times in 4.12, the most of any section

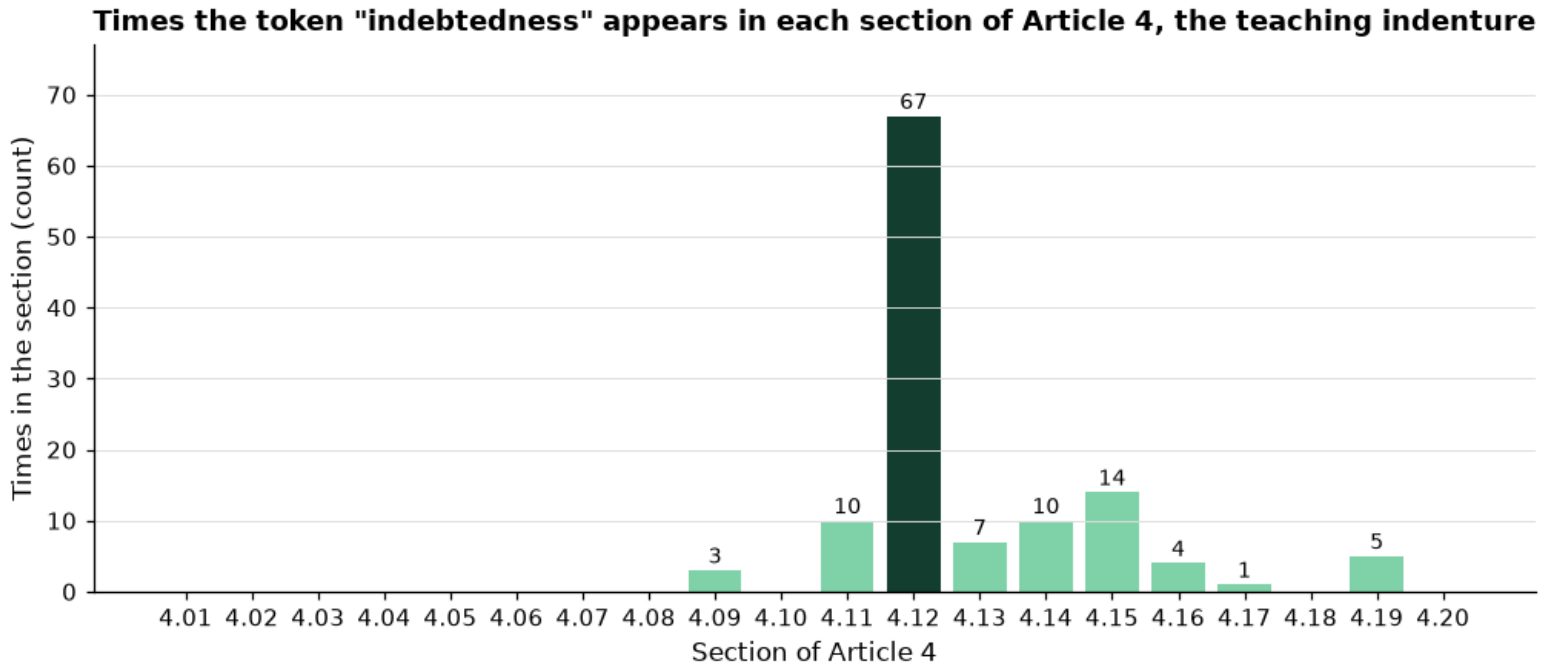
Section	and	Cosine
4.10	4.11	0.5688
4.12	4.15	0.5330
4.12	4.16	0.4663

What 4.10 and 4.11 share most

offer, notes or, or portions, portions: each section sets out an offer to the holders.

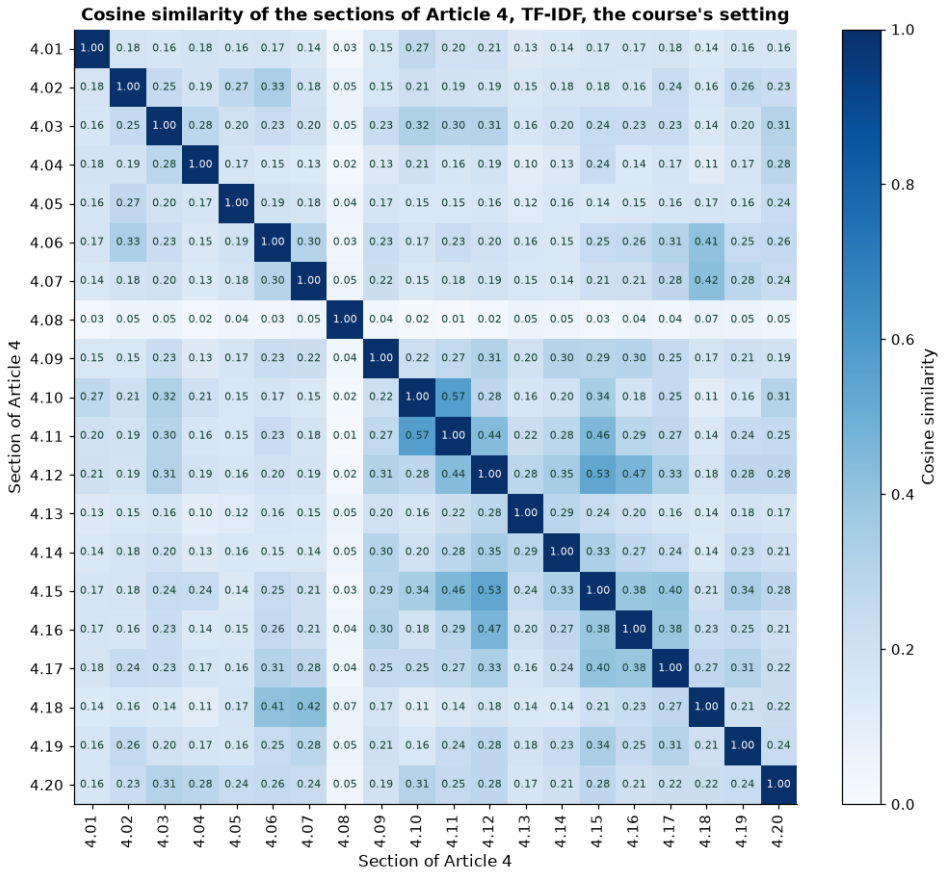
Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

Indebtedness, section by section



67 times in 4.12. The idf is the same in every section: the count makes 4.12's weight the largest.

Every pair of sections

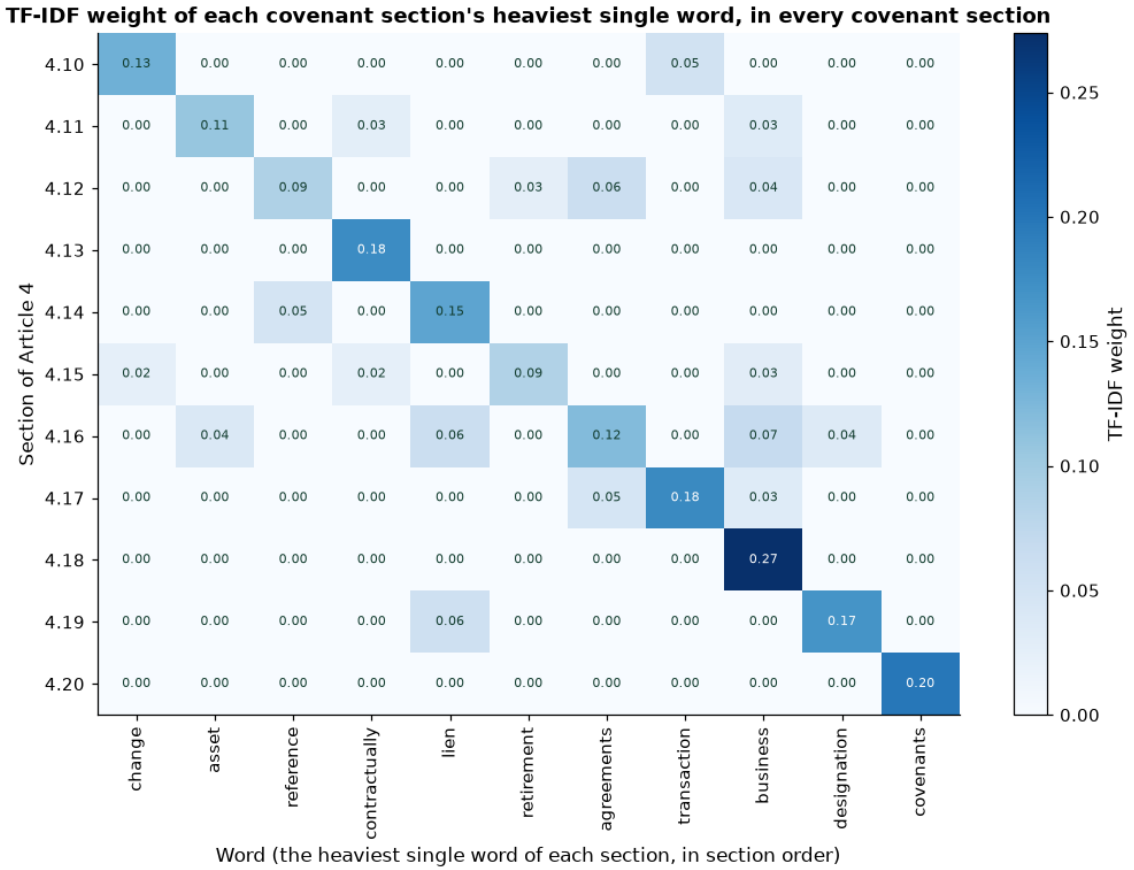


The five most alike pairs

4.10 and 4.11	0.5688
4.12 and 4.15	0.5330
4.12 and 4.16	0.4663
4.11 and 4.15	0.4554
4.11 and 4.12	0.4353

All five sit among 4.10 to 4.16. 4.08, reserved, is pale all the way across. Shared wording, nothing more.

Each section's own word



A document-by-term table

Rows: sections 4.10 to 4.20. Columns: each section's heaviest single word. Cells: its weight.

Each word weighs most in its own section; 93 of the 121 cells are 0.

What a bag of words loses

```
Leverage is less than 3.0 and coverage is not less than 2.0.
Leverage is not less than 3.0 and coverage is less than 2.0.
```

1.0000

cosine as bags of words

1.0000

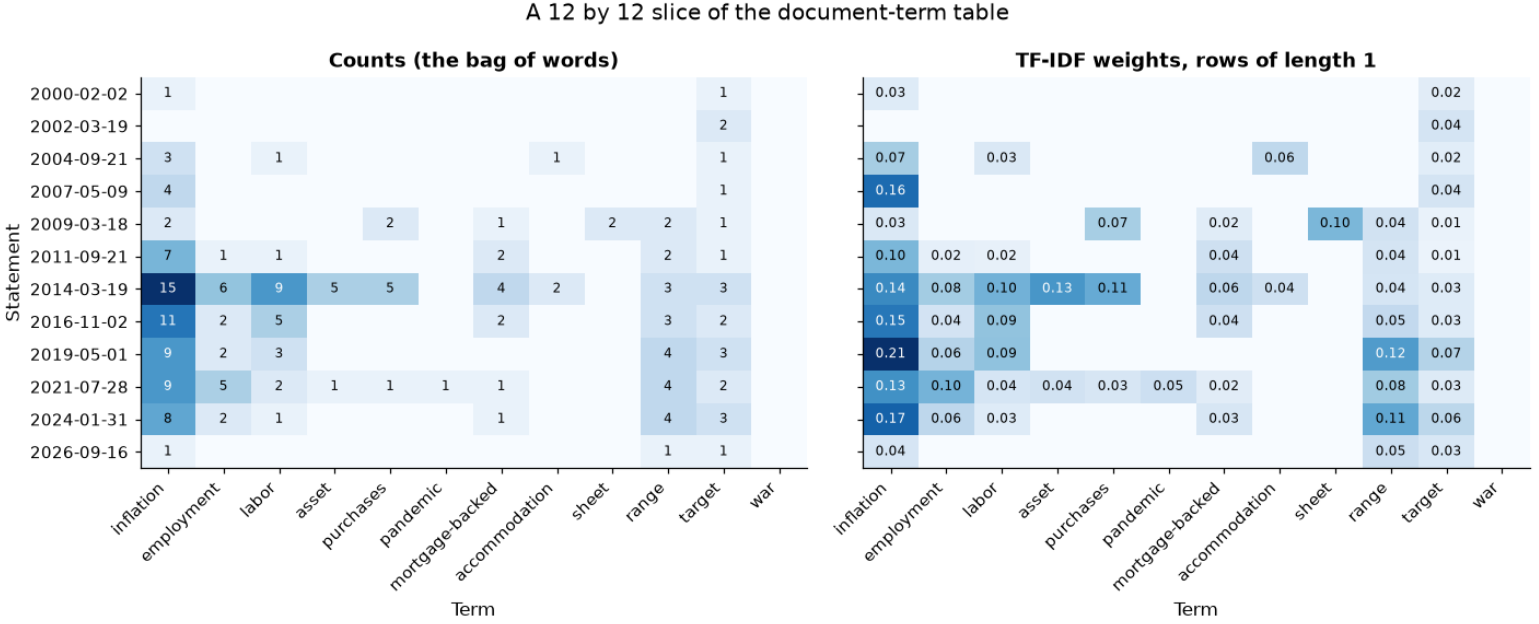
with bigrams too

0.9512

with trigrams

Same words, opposite tests. A bag of words indexes what a text is about, not what it says.

The table as a picture



85 of 144

cells of the slice are 0

The largest count is not the largest weight: each row is divided by its own length, and a rarer word weighs more for the same count.

A blank cell is a word the statement does not use.

How a vectorizer works

The problem	About twenty settings, and no way to know which ones matter until you see what each changes
Stage 1	Terms: the token pattern, lower-casing, stop words, and word pairs
Stage 2	Keep some: a term must be in at least min_df documents and at most max_df of them
Stage 3	Weight each cell: counts or presence, TF-IDF with or without the log, the row scaling
What comes out	The same kind of table at every setting; four settings move it most

Every setting of TfidfVectorizer

ngram_range (default (1, 1); course (1, 2))

The lengths of word runs counted as terms. (1, 1): single words.

(1, 2) adds pairs of neighbours; every step up multiplies the columns and keeps a little word order.

max_df (default 1.0; course default)

Drops a term found in more than this share of the documents (or this number). 1.0: drops none.

Lower drops the commonest words, by the data's own counts rather than a list.

min_df (default 1; course 2)

Drops a term found in fewer documents than this. 1: drops none.

Higher drops rare terms: far fewer columns, and the one-document words go first.

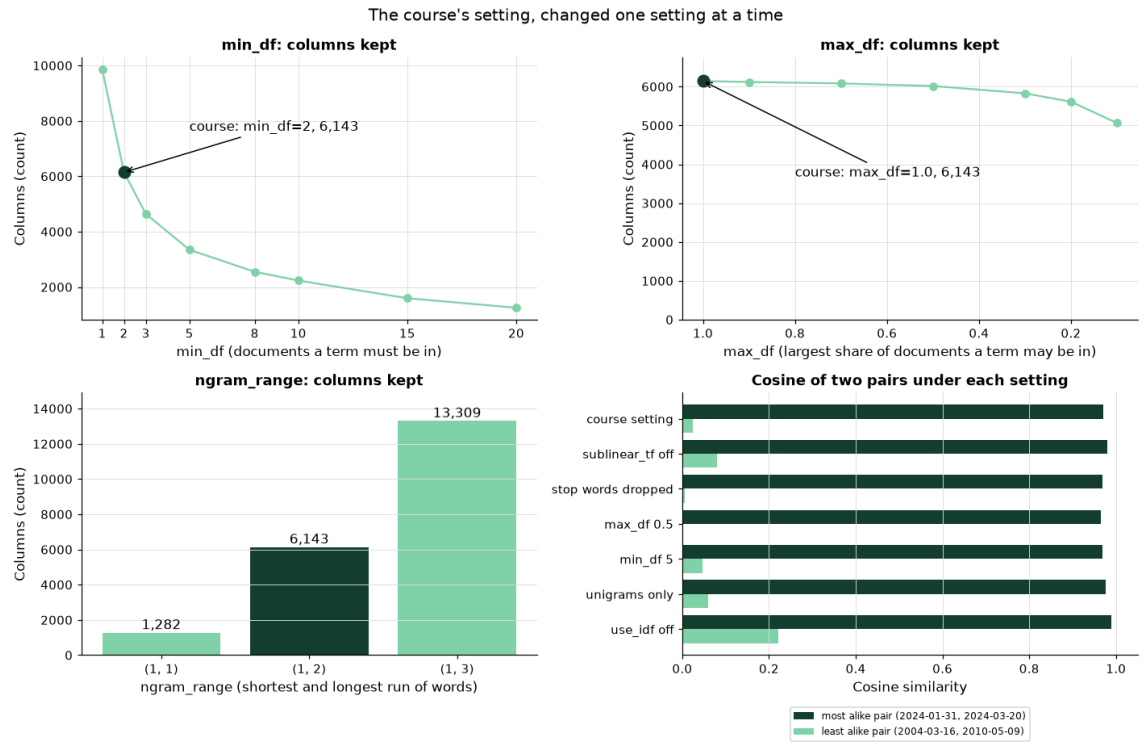
sublinear_tf (default False; course True)

Replaces a count with 1 plus its natural log. False: the count.

True keeps a word said ten times from weighing ten times as much; 'the' stops leading section 2.3's first statement.

21 settings in four groups, read from the installed library. The other 17 stay at their defaults, or are set once.

Four settings, one at a time



3,726

columns removed by min_df 1 to 2

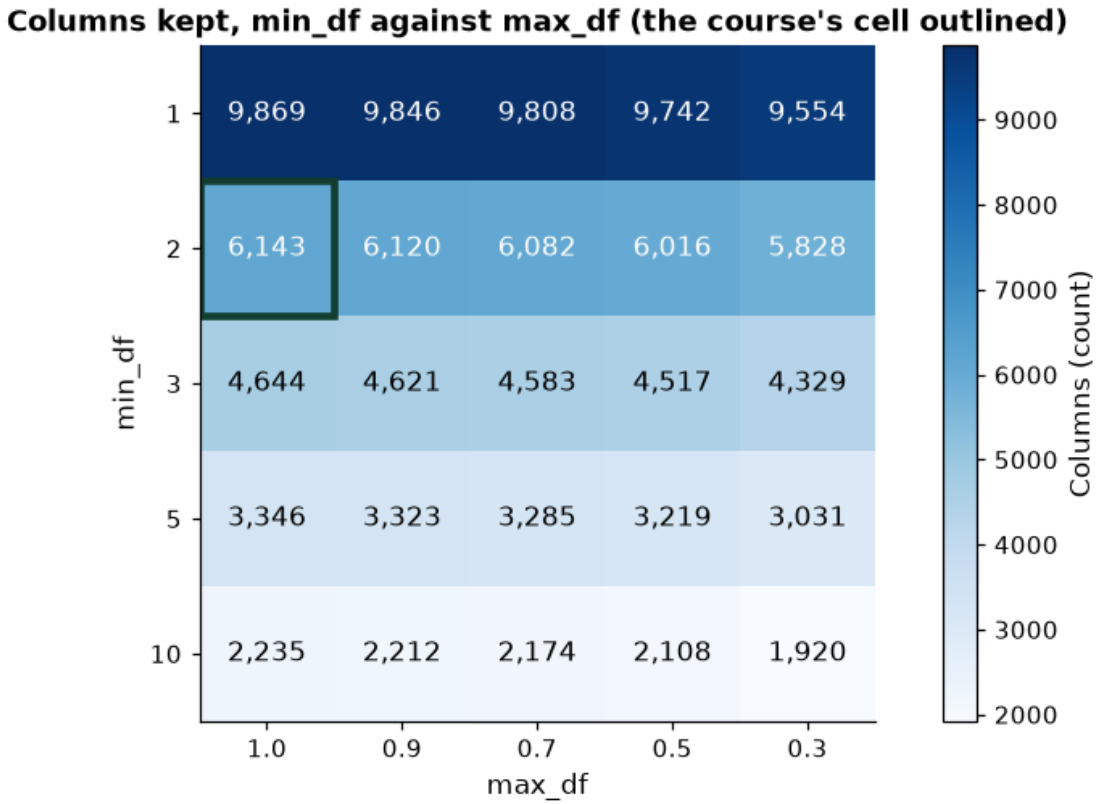
0.2217

least alike pair, use_idf off

norm=None leaves the cosine unchanged: the cosine divides by the lengths itself.

Read each panel as one setting changed from the course's.

Two settings together



6,143

columns at the course's cell

1,920

columns at min_df 10, max_df 0.3

min_df takes thousands of columns, max_df dozens. The grid shows size, not quality.

Excel side by side

Excel side by side (tested in Excel)

`=COUNTIF(A2:A227,"*inflation*")`	205, as Python
`=COUNTIF(A2:A227,REPT("a",256))`	#VALUE!; 255: 0
`=LN((1+226)/(1+J2))+1`	within 1e-12 (123)
`=I2*K2`	TF times IDF
`=L2/SQRT(SUMSQ(L$2:L$124))`	within 1e-12
`=SUMPRODUCT(P1:P6143,Q1:Q6143)/(SQRT(SUMSQ(P1:P6143))*SQRT(SUMSQ(Q1:Q6143)))`	0.9699

No tokenizer and no TF-IDF in Excel: once the counts exist, it checks every step after them.

Save the day: read the diff

1. Morning

Commit textkit as day 1 left it

2. Lesson

Four functions, four appends

3. Diff

2 files, 150 lines added, none removed

4. Commit

Today's two files

```
d2b0d6f Add term_counts, idf, tfidf_by_hand, cosine_similarity_matrix, 14 tests
6a286ee Start of day 2: textkit as day 1 left it, 7 tests
```

pytest: 14 passed. Fragments only append: a minus line is the first thing to read.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: IDF as log10(n / df): 5 terms weigh 0, and the top term moves from "the" to "pronounced".

Code that runs is not the same as code that is right.

What you can do now

- Count tokens into a bag of words, by hand and with CountVectorizer
- Compute scikit-learn's smooth IDF and TF-IDF by hand, to 1e-12
- Measure how alike two documents are with a cosine
- Find what sets a year, or a covenant section, apart
- Do every step after the counts in Excel

Week 1, Day 3 Classifying 8-K items with TF-IDF